CertaRig engineering agent commissioning

Capstone implementation report draft

Chintan Dedhia

BSc Computer Science Online Mode

Roll number and institution details to confirm

Academic year and current supervisor to confirm

15 November 2026

Draft status

This report records a locally tested software increment. It is not a final university submission. Genuine provider and container qualification, physical evidence or an approved scope amendment, institutional metadata and a recorded demo remain outstanding.

Declaration

The institution requires a student declaration of original work. The student must review and complete that declaration before submission. No signature or submission approval is supplied in this draft.

Abstract

Engineering laboratories need a repeatable way to describe equipment, inspect measurements, review proposed software behavior and retain evidence. A language model can assist with these tasks, but its instructions cannot establish equipment permissions, measurement validity or physical readiness. CertaRig separates conversational assistance from application identity, local acquisition, deterministic checks and independent human review. The physical bench remains a reference installation for a software platform intended to support other explicitly mapped engineering setups.

This implementation extends the local platform with the Session Journal interface, persistent conversations, revisioned agent drafts, exact-version teams, background evaluation, isolated-analysis interfaces and portable evidence. A new bench-description service records operator-declared equipment and channel facts, supporting references, unresolved questions and profile bindings. Deterministic support reports distinguish missing mapping evidence, unfamiliar experiments that permit analysis only, and named procedures that still require qualification. Guided forms and an advanced editor expose these records without granting control authority.

The implementation uses Python, SQLite, modular JavaScript and existing provider-neutral agent sessions. Local regression met the ninety percent coverage requirement; browser checks exercised the Journal and legacy Edge workflows with labelled test dependencies. Genuine identity and model qualification, the pinned Sandbox image and seven real-container cases remain outstanding. No new physical measurements or hardware operation are claimed. The resulting software supports reviewable commissioning work while keeping unsupported equipment, missing evidence and unavailable isolation visible. Final submission requires completion of the remaining qualification gates and supervisor resolution of the promised physical evidence.

Contents

Chapter 1 Introduction

Chapter 2 Implementation details

Chapter 3 Testing validation and results

Chapter 4 Execution and deployment details

Chapter 5 Project execution evidence

Chapter 6 Conclusion and future work

References

Appendix A User manual

Appendix B Installation guide

Appendix C Source and demonstration links

List of figures

Figure 1 Session Journal with deterministic prototype data

List of tables

Table 1 Selected acceptance cases

Abbreviations

API Application Programming Interface; CSV Comma Separated Values; HIL Hardware in the Loop; OIDC OpenID Connect; SDK Software Development Kit; UI User Interface.

Chapter 1 Introduction

Project overview

CertaRig supports engineers who commission or revalidate an already mapped test setup. The customer describes the intended experiment, identifies available signals and tools, inspects an agent proposal, evaluates its behavior and obtains independent review before bounded observation. The software retains the definitions, input profiles, traces and results needed to inspect what happened. The local reference bench demonstrates this workflow; it does not define the only type of customer equipment the platform could eventually support.

Problem and motivation

A bench description often mixes equipment labels, channel addresses, measurement units, calibration assumptions and informal operating instructions. A connection can return numbers without establishing whether those numbers describe the intended physical quantity. An agent can produce a plausible explanation without possessing a valid calibration record or current execution permission. These distinctions become difficult to inspect when conversations, configuration files, approvals and recordings live in separate places. The commissioning problem therefore includes provenance, authorization and reproducibility as well as interface design.

Supported equipment and uncertainty

The implementation starts with known pressure and flow profiles and a separate thermal simulator. MQTT, serial JSON-line telemetry and Modbus TCP provide observation interfaces through operator-enrolled local connectors. A protocol name alone does not identify the experiment, establish a safe operating range or prove that a device was physically commissioned. Unfamiliar equipment produces a support report and integration checklist. It does not inherit a trusted rehearsal from the thermal or pressure examples.

Objectives

The software objective is one inspectable journey from an experiment goal to a reviewed agent, bounded observation and independently verifiable evidence. The interface objective is to retain engineering context while reducing the old collection of forms into a persistent journal, an inline

proposal and a contextual inspector. The validation objective is to demonstrate success, refusal and recovery while preserving identity, asset scope, exact-version binding and restart behavior. The academic objective is to map actual evidence to each signed Phase 3 mission and the institution's assessment criteria.

Implementation scope

This increment remains local and observation-only. It reuses AgentSession, the current tool registries, reviewed profiles, evaluation, managed deployment and evidence verification. The approved Session Journal direction replaces the product's presentation while a classic route preserves the earlier interface during migration. New hardware, public hosting, unrestricted endpoint discovery, arbitrary connector installation and physical actuation are outside this software change. The current demonstration-accessory budget is below INR 3000; no purchase or operation is authorized by this report.

Authority and report organization

Application identity, model sign-in, bench grants and local equipment authority remain separate. An author cannot independently approve their own proposal. A description, analysis result or successful process exit does not become qualification. Chapters 2 and 3 explain implementation and validation. Chapter 4 describes local execution. Chapter 5 records actual project evidence and open institutional questions. Chapter 6 identifies remaining work. This is an implementation draft; it does not replace an unsigned declaration, a supervisor decision or a final submission approval.

Chapter 2 Implementation details

Architecture and data flow

Studio sends authenticated requests to the local platform. The platform derives organization and role from its session store, checks current asset grants and manages versioned definitions, tasks and reviews. Local Edge acquisition remains separate from the agent runtime. The Sandbox broker is a separate isolation boundary for submitted Python and SPICE. Its containers cannot obtain equipment credentials or issue trusted receipts by writing a pass file. The observation runtime checks its lease and reviewed bindings before using authorized connectors.

Session Journal

The new shell has persistent navigation, a central session and a contextual inspector. A focused dialog contains advanced agent editing. Dedicated destinations expose agents and teams, benches, Sandbox, evidence, deployments, connectors and workspace settings. The current definition appears before lengthy conversation history so the next inspection action remains visible. The inspector separates proposal, activity and source observations. Desktop, tablet and phone layouts change panel presentation rather than shrinking a desktop screenshot.

Persistent drafts and exact versions

Drafts retain conversations and optimistic revisions in SQLite. Updates require the expected revision. Concurrent changes preserve local input and offer a comparison, loading the server revision or saving local edits into a separate draft. Explicit immutable save creates a version; editing that version creates another version. Saved session URLs restore recorded draft or version provenance. A standalone version without an originating conversation displays that absence instead of manufacturing history. Queued saves and generation guards prevent late responses from replacing a newly selected draft.

Agents and teams

Individual agents use the existing provider-neutral AgentSession. A team pins a supervisor and one to three evaluated specialists on one bench. Delegation stays sequential and one level deep. The complete exact-version team requires independent review; a separate deployment review is

not invented for every member. Shared limits remain at most eight model calls per assessment, ten assessments and a fifteen-minute lease. Native provider delegation cannot expand CertaRig permissions.

Evaluation and bounded deployment

Collection and trusted qualification are separate background tasks with status, ordered events and cancellation. Evaluation records nominal, stale-data, wrong-unit, forbidden-action and cross-asset cases. A current trusted receipt and an independent Reviewer are prerequisites to a reviewed observation deployment. Starting is a separate action. Runtime health, provider activity, lease expiry and revocation remain inspectable. Revocation across navigation does not depend on keeping the original deployment page open.

Bench descriptions and support assessment

Bench Specification version 1 stores owner, organization, optional linked asset, revision, public-content hash, declared facts, provenance and history. Unlinked descriptions are private to their owner. Linking requires a current bench grant. The schema permits incomplete facts so missing identity, unit, source, scaling, calibration, freshness and operating limits can become explicit questions. A reviewed description is bound to its exact revision and content hash and grants no execution authority. Changes preserve prior facts and review history while removing the current review.

Quantity and unit logic

The deterministic registry distinguishes quantity kinds as well as units. Pressure retains gauge, absolute or differential reference context. Absolute temperature conversion has an offset; temperature differences use a separate quantity. Supported electrical observations describe V, A, mA and digital state without creating a qualified electrical procedure. Zero is a valid value. Missing, contradictory or unsupported information does not become a numeric default. Support reports return transformation metadata, not fabricated measured samples.

Sandbox tools and recorded analysis

Users can author Python and SPICE tools with input and output contracts, declared tests and immutable source. Testing and independent review bind source, image and inputs. Reviewed tools run through the isolated broker only. The Journal exposes source, parameters, job state, cancellation and safe artifact inspection. Completed-run CSV imports retain unverified-origin provenance and can be selected for analysis. Tool input, recording selection and editing focus are retained during polling. Missing isolation refuses execution without a host-process fallback.

Connectors and evidence

MQTT subscribes for observation, serial ingests bounded JSON lines without application commands, and Modbus uses read-only functions. Enrollment metadata exposes registered names and modes without endpoints or credentials. Mapping review yields an operator-installation proposal rather than applying Edge configuration. The evidence library exposes readable findings, raw observations, tool traces, bundle downloads and offline integrity verification. Existing evidence formats remain supported. Hashes prove consistency with the supplied package; they do not independently prove physical authenticity.

Implementation evidence

The source is on the product/session-journal-2026-11-14 branch, based on commit 13bd409a1ec045df986b1fc1aa8e129562a3ddee. Relevant modules include platform/bench_specs.py, platform/workbench_api.py, the platform client and schemas, and Studio's journal, workbench and bench-description modules. The selected visual system uses local Geist and IBM Plex Mono assets with their supplied licences. The classic product route and legacy Edge Studio remain available for feature-parity checks.

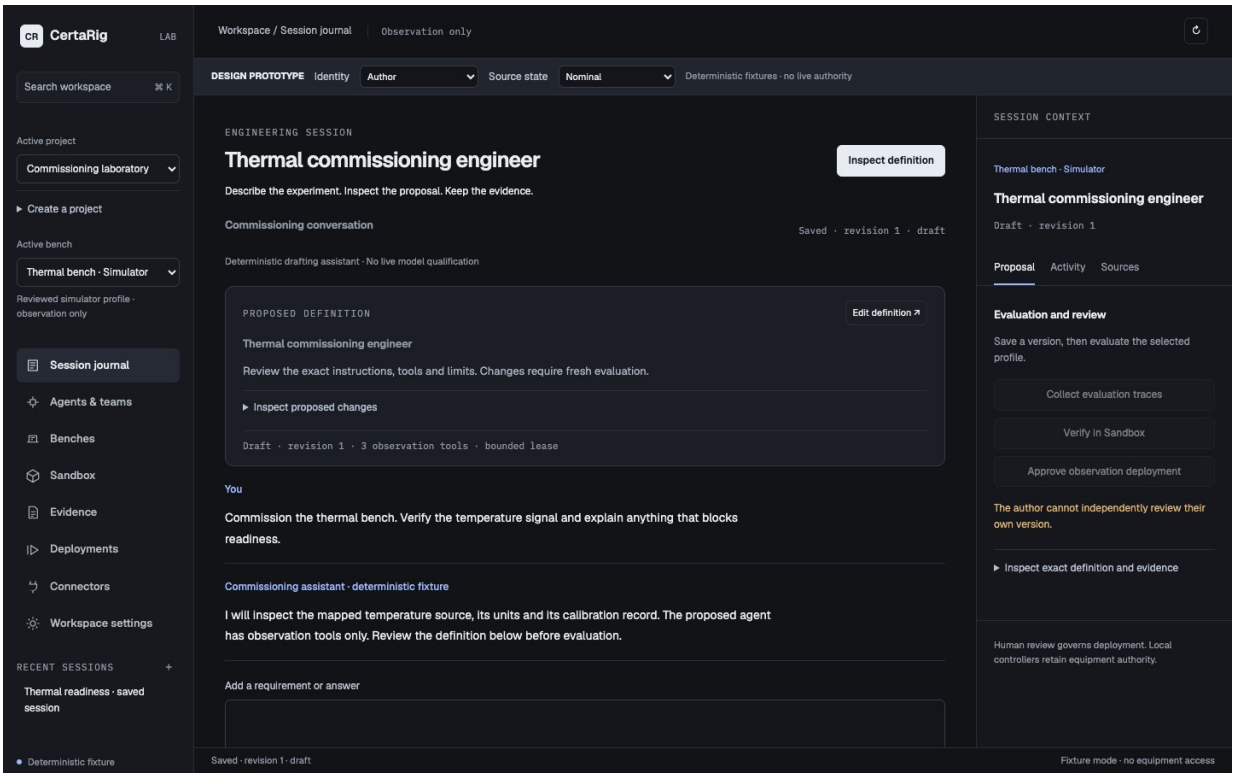


Figure 1 Session Journal with deterministic prototype data

Chapter 3 Testing validation and results

Test strategy

Validation separates contract and authorization tests, property and scenario checks, browser behavior, resource preflight and genuine runtime qualification. Deterministic providers, test identities and test engines are labelled fixtures. Their passing results establish software behavior under those inputs, not real account ownership, external device performance or container isolation. No generated code was executed directly on the host.

Measured software results

The local full regression met the ninety percent coverage gate at 90.63 percent. Lint passed and mypy reported no issues in 120 source files. Forty new description and advanced-draft tests passed. Eleven platform browser checks exercised the new Journal and retained classic interface; ten legacy Edge browser checks passed. The simulated-hour soak check passed. These figures describe the recorded local test environment and should be regenerated after relevant changes. No physical response time, model accuracy, general bench-recognition accuracy or commercial reliability figure was measured.

Required cases

The new description tests cover missing and contradictory facts, quantity and unit context, valid zeros, opaque malicious manual text, revision conflicts, unlinked ownership, cross-organization refusal, revoked asset grants, imported-recording provenance and independent description review. Existing suites cover mismatched or reused approvals, profile and model binding, restart invalidation, team budgets, revocation, forged analysis results and incomplete or corrupted evidence. The browser journey saves and reopens requirements, evaluates, inspects sources, obtains independent fixture review, starts observation and revokes from another view.

Design and usability verification

The selected Session Journal received an independent visual review of 8 out of 10 without a blocking layout defect. The implemented refinements strengthen the next action, shorten prose lines and reduce repeated mobile status text. An independent usability walkthrough completed

the prototype lifecycle and identified insufficient readiness explanation and incomplete exact-definition evidence. The prototype now includes the relevant result, calibration reference, complete instructions and limits. Automated checks cover four viewport widths, dialogs, search, reload recovery and horizontal overflow. Native mobile keyboard behavior remains unverified.

Open qualification gates

Genuine Dex two-account login and live Codex qualification remain outstanding. Sandbox preflight reported elevated macOS memory pressure and no pinned image; the seven real-container tests remain unrun. The system must refuse untrusted execution under this condition. A genuine local MQTT broker check and real external serial or Modbus equipment qualification remain distinct from in-process or loopback fixtures. Physical calibration, measured feedback and protected actuation have no new results in this increment.

Interpretation

The evidence supports a locally tested software workflow and honest handling of unsupported equipment. It does not support a claim that any connected experiment can be commissioned automatically. Further support requires an operator-enrolled transport, reviewed mappings, a suitable procedure and model suite, relevant calibration evidence and exact-profile qualification. Changes to Python source or the runner image can invalidate Sandbox policy binding, so final runtime qualification must follow the final relevant code changes.

Table 1 Selected acceptance cases

ID	Description	Input	Expected result	Status
BS01	Revision conflict	Old revision	Refuse overwrite	Passed
BS02	Incomplete mapping	Unknown units	Explicit blockers	Passed
BS03	Unfamiliar procedure	Electrical V	Analysis only	Passed
BS04	Scope loss	Revoked grant	Access refused	Passed

ID	Description	Input	Expected result	Status
UI01	Draft recovery	Saved session URL	Restore actual conversation	Passed
UI02	Independent review	Two test identities	Author refused and Reviewer allowed	Passed
RT01	Container isolation	Pinned image absent	Refuse execution	Unrun
HW01	Physical feedback	No new trial	No result claimed	Unrun

Chapter 4 Execution and deployment details

Local execution

Use a source checkout with Python 3.11 or later and the documented dependency extras. Studio uses modular browser JavaScript and does not require a frontend framework migration. Follow the platform setup guide to initialize a new private demonstration installation, provision actual Author and Reviewer identities, start the simulator Edges and the separate observation runtime, and configure the qualified Sandbox broker. Initialization must not overwrite an existing installation. Open the local platform at its configured loopback origin; /classic retains the previous shell.

Demonstration

The five-minute demonstration shows a supported thermal task, an exact agent proposal, a deliberate stale-source or unit refusal, corrected evaluation, independent review, observation, revocation and evidence inspection. When genuine Sandbox qualification is unavailable, show the refusal and label any design demonstration as a fixture. Do not substitute fixture artifacts for real container output. The supplied demonstration script reserves five minutes within the institution's fifteen-minute presentation.

Release boundary

This is a supported-scope local release candidate. Public hosting, multi-tenant commercial security, arbitrary remote endpoints and physical certification are not established. The clean-installation procedure must be reproduced from the release source, and current runtime qualification blockers must remain visible. A final demo recording link has not been produced in this coding session and must be added after an actual recording.

Chapter 5 Project execution evidence

Version control and work record

The GitHub repository is https://github.com/shadybrook/certarig-platform. The implementation uses an isolated worktree and a separate Session Journal branch. The frozen Phase 3 installation and earlier artifacts are preserved. The dated validation folder contains software checks, browser records, preflight results and the current qualification ledger. A branch or commit is evidence of source changes, not proof of deployment or successful physical operation.

Recorded progress

On 14 November 2026, the student selected Session Journal A and requested its execution. Work on 14 and 15 November established the baseline, built the clickable prototype, completed independent visual and usability reviews, connected the Journal to existing APIs, added versioned bench descriptions and ran local validation. The updated progress record contains these actual activities. Later dates remain plans until work occurs. No supervisor meeting, attendance or remark is inferred from code activity.

Supervisor and institutional decisions

The signed Phase 3 report is the approved historical baseline. No new supervisor response was received during this implementation. The current supervisor assignment, final university deadline and submission process remain unconfirmed. The working software checkpoint is 20 November 2026. The latest software-first and under-INR-3000 accessory direction must be reconciled with promised measured-feedback and protected-actuation evidence through real results or a genuine approved scope amendment.

Chapter 6 Conclusion and future work

Implemented outcome

CertaRig now presents commissioning work through a persistent Session Journal and focused authoring, review, Sandbox and evidence views. The bench-description layer records what is known and missing and provides deterministic support reports. Local tests exercise the critical software boundaries while preserving observation-only operation and independent review. The product remains extensible through supported connector and experiment packs.

Remaining work

Complete genuine identity and live-provider qualification, build and qualify the exact Sandbox image when resource preflight passes, run the seven real-container cases and qualify relevant connectors with actual equipment. Verify the mobile keyboard and complete a genuine demonstration recording. Resolve each signed mission with evidence or supervisor-approved amendment before final academic submission. Model training, additional SCPI or OPC UA connectors, hosted organizations and wider experiment packs are later increments rather than hidden claims of this release.

References

[1] Signed CertaRig Phase 3 report pages 18 to 21 provided by the student.

[2] Institutional Document format and Capstone Rubrics supplied by the student.

[3] Institutional Capstone Project 15 Min Demo Template supplied by the student.

[4] CertaRig platform source and local validation records on product/session-journal-2026-11-14.

[5] T3 Code product reference https://t3.codes/ viewed for persistent conversation and inline review inspiration.

[6] Colima documentation https://colima.run/docs/commands/ and Docker resource, networking and seccomp documentation referenced in the Sandbox implementation guide.

Appendix A User manual

Select a permitted bench and describe the commissioning goal. Inspect the proposed instructions, tool scope and limits. Save a draft and resolve any missing-information prompts. Save an immutable version, collect the evaluation and inspect its raw traces. Obtain genuine Sandbox qualification before independent deployment review. Start bounded observation separately and monitor its health. Revoke when finished. Export evidence and run the offline verifier. For an unfamiliar bench, open Benches and create a description; resolve support questions without treating description review as execution authority.

Appendix B Installation guide

Clone the release source and create an isolated Python environment. Install the documented development and platform extras; install Studio test dependencies only when running browser checks. Follow docs/product/AGENT_LIFECYCLE.md for actual identity provisioning, local simulator services, managed model sign-in, qualified Sandbox and the observation runtime. Run the documented preflight before preparing a runtime. Do not overwrite an existing installation,

switch Docker contexts globally or weaken isolation. Source checkout execution is required for Studio assets in this release.

Appendix C Source and demonstration links

Source repository: https://github.com/shadybrook/certarig-platform. Branch: product/session-journal-2026-11-14. Validation records: docs/product/validation/session-journal. Demonstration video link: pending an actual attended recording. Supervisor remarks: no new remarks received. Final institution deadline and submission process: unconfirmed.